

DisloCluster User Manual

Installation, Build, and Simulation Guide
Coupled Cluster Dynamics – Dislocation Dynamics of Irradiated Zirconium

Nasr M. Ghoniem

August 16, 2026

Repository: <https://github.com/Ghoniem/DisloCluster>

Abstract

DisloCluster simulates radiation effects in zirconium by coupling a zero-dimensional (0-D) reduced cluster-dynamics model of point-defect and dislocation-loop evolution (ZrMicro) to a three-dimensional, spatially resolved cluster-dynamics / dislocation-dynamics code (MoDELib3, a fork of MoDELib-fullCD). This manual takes a user from a fresh clone of the repository on a new Windows machine to a completed simulation: installing the prerequisites (Section 2), creating the Python virtual environment (Section 3), compiling the two C++ codes (Section 4), and running the three kinds of simulation the repository supports — the standalone 3-D single-crystal test (Section 5), the 0-D ZrMicro model (Section 6), and the coupled 0-D $\leftrightarrow$ 3-D dose march (Section 7). Section 8 explains where to change physical and numerical parameters, Section 9 collects the known failure modes and their fixes, and Section 10 describes the provenance system that makes every run reproducible. Appendix A is a condensed architecture reference, Appendix B summarizes the physics added relative to the upstream MoDELib-fullCD code, and Appendix C is a one-page command summary.

Contents

1	Introduction	3
1.1	What DisloCluster does	3
1.2	The three ways to run it	3
1.3	Repository layout	4
1.4	Conventions used in this manual	5
2	Installation	5
2.1	Prerequisites	5
2.2	Step 1: Windows tools	6
2.3	Step 2: clone the repository	6
2.4	Step 3: WSL and Ubuntu	6
2.5	Step 4: SUNDIALS 7.1.1	7
3	The Python virtual environment	7
3.1	Create the venv and install dependencies	7
3.2	Register the Jupyter kernel	7
3.3	Verify path resolution	7

4	Building the two codes	8
4.1	The 0-D ZrMicro solver (Windows, native)	8
4.2	The 3-D MoDELib3 code	8
4.3	Verify the builds	9
5	First simulation: the 3-D single-crystal test	9
5.1	The case	9
5.2	Quick start: figures in minutes	10
5.3	Running the solve	10
5.4	Reading the figures critically	11
6	The 0-D ZrMicro simulation	11
6.1	Running it	11
6.2	Inputs: the Excel workbook	11
6.3	The state vector	12
6.4	Solver configuration	12
7	The coupled 0-D $\leftrightarrow$ 3-D simulation	12
7.1	The idea	12
7.2	Two options for the slow step	13
7.3	Starting from pristine material	14
7.4	The notebook's three control blocks	15
7.5	Meshing, and the materialized simulation directory	15
7.6	Fast-solve backends	15
7.7	Running the march	16
7.8	Outputs	16
8	Changing things	17
9	Troubleshooting	18
10	Reproducibility and provenance	19
A	Architecture reference	19
A.1	The governing split	19
A.2	Key components	21
A.3	The three interfaces	21
A.4	Unit systems	21
A.5	Call graph	21
A.6	Anisotropic diffusion and the discrete handoff	21
A.7	Known non-correspondences between the 0-D and 3-D models	23
B	Changes relative to upstream MoDELib	23
C	Quick reference	24

1 Introduction

1.1 What DisloCluster does

Irradiated zirconium develops a microstructure of prismatic $\langle a \rangle$ interstitial loops and basal $\langle c \rangle$ vacancy loops, fed by the point defects (vacancies, single interstitials, di- and tri-interstitials) that displacement cascades produce. The physics splits into two widely separated time scales: the *mobile* point-defect species relax to their local sink environment in milliseconds, while the *immobile* loop populations evolve over dose steps of order 10^7 s. DisloCluster exploits that split with an operator-split quasi-steady-state (QSSA) scheme: within each dose step the fast mobile species are solved as a steady reaction–diffusion field, and the slow loop populations are then marched in dose against the frozen mobile field.

The repository contains two codes that realize this physics at different levels of spatial resolution, plus the coupling layer between them:

- **ZrMicro** (`ZrMicro/`) — the 0-D reduced model: 19 coupled ODEs (12 physical species, 6 conservation accumulators, and the evolving network dislocation density ρ_N), integrated with SUNDIALS CVODE through a compiled C++ solver, orchestrated and post-processed in Python.
- **MoDELib3** (`MoDELib3/`) — the 3-D spatially resolved cluster-dynamics / dislocation-dynamics code: a finite-element solver for the steady mobile boundary-value problem and a pointwise dose march for the four loop families, run by the `DDomp` executable.
- **The coupling layer** (`dislocluster_code/coupling/` and the notebook `Simulations/run_simulation.ipynb`) — exchanges the per-node immobile field and the steady mobile field between the two codes over a shared dose schedule.

1.2 The three ways to run it

Everything in this manual funnels into one of three simulation types, and it is worth fixing them in mind now:

1. **The 3-D single-crystal run** (Section 5) — a standalone MoDELib3 simulation of a $1\ \mu\text{m}$ cubic grain under irradiation, driven by a shell script. No Python is involved in the solve. This is the recommended first test on a new machine, because the repository ships with the results of a completed solve (`ev1/`), so the post-processing can be exercised in minutes, before the hours-long solve is attempted.
2. **The 0-D ZrMicro run** (Section 6) — the reduced model alone, driven by the notebook `ZrMicro.ipynb`. Fast (minutes), and the source of the fitted parameter set.
3. **The coupled 0-D $\leftrightarrow$ 3-D march** (Section 7) — the operator-split coupling, driven by the notebook `Simulations/run_simulation.ipynb`. Python owns the dose loop, MoDELib3 solves the fast mobile field, and the ZrMicro C++ integrator marches the immobile state at every quadrature point. This is the normal entry point for new work; the older `coupled_0d_3d_ZrMicro.ipynb` carries its own inline copy of the march and is kept only for reference.

Note. **Updated (14 August 2026).** The Python was gathered into one installed package, `dislocluster_code/`, and simulations moved to `Simulations/`, which now holds `input/` and `output/` beside the notebooks the way `ZrMicro/` used to. `ZrMicro/py_utils/` survives as a shim: every former module is an alias for its new home, so from `py_utils import paths` and `python -m py_utils.X` still work. Runs made before the move stay under `ZrMicro/output/` and are still found, because `paths.find_runs()` searches every output root.

1.3 Repository layout

The repository is self-contained: nothing outside the repository root is required except the Python interpreter, SUNDIALS, and (on Windows) WSL. The layout is:

<code>DisloCluster/</code>	<code><- repository root (.dislocluster_root marker)</code>
<code> - .DisloClusterVenv/</code>	Python 3.14 environment (you create this)
<code> - Libraries/</code>	SUNDIALS install location (you create this)
<code> \- sundials-7.1.1/</code>	
<code> - requirements.txt</code>	pinned environment lock
<code> - pyproject.toml</code>	the package (pip install -e .)
<code> - dislocluster_code/</code>	ALL Python lives here
<code> - paths.py</code>	single source of truth for every location
<code> - config.py</code>	SimulationConfig: the notebook's six dicts
<code> - driver.py</code>	prepare / march / report
<code> - build.py</code>	ensure_zrmicro_solver, ensure_modelib
<code> - zerod/</code>	the 0-D model chain
<code> - staging/</code>	mesh, input files, seed, bootstrap
<code> - coupling/</code>	the operator split, checkpoint, progress
<code> - post/</code>	figures, movies, discrete loops, reports
<code> \- studies/ fitting/ legacy/</code>	comparison drivers, fits, superseded code
<code> - Simulations/</code>	WHERE SIMULATIONS ARE RUN
<code> - run_simulation.ipynb</code>	<code><- the normal entry point</code>
<code> - postprocess.ipynb</code>	re-render an existing run
<code> - input/</code>	the Excel parameter workbooks
<code> \- output/</code>	timestamped run directories
<code> - Gmsh/</code>	generate_mesh.py + meshes/ cache
<code> - Docs/</code>	
<code> - DisloCluster Manual/</code>	this manual
<code> \- Formulation/</code>	formulation documents + build_modelib.sh
<code> - ZrMicro/</code>	0-D side, C++ only now
<code> - code/</code>	solver.cpp + the older notebooks
<code> - ZrMicro.ipynb</code>	0-D run + parameter identification
<code> \- coupled_0d_3d_ZrMicro.ipynb</code>	superseded by Simulations/
<code> - cpp_utils/</code>	C++ RHS + CMake build (SUNDIALS CVODE)
<code> - py_utils/</code>	compatibility shim -> dislocluster_code
<code> - input/</code>	older workbook copies (not read)
<code> - build/</code>	compiled solver.exe (created by the build)
<code> \- output/</code>	runs made before Simulations/output/ existed
<code>\- MoDELlib3/</code>	3-D side (fork of MoDELlib-fullCD)
<code> - Library/Materials/</code>	Zr3d_ghoniem.txt is the coupled material
<code> - tutorials/</code>	staged simulation cases + the reference case
<code> \ - build/</code>	DDomp + pyMoDELlib (built inside WSL)

Nothing in the repository hard-codes an absolute path. The single source of truth for every location is the module

`dislocluster_code/paths.py`

which finds the repository root by walking up to the `.dislocluster_root` marker, so the repository can be cloned or moved anywhere. Three optional environment variables override the defaults: `DISLOCLUSTER_ROOT` (repository root), `MODELIB_ROOT` (a `MoDELib3` checkout outside the tree), and `MODELIB_BUILD` (its build directory).

Note. Resolved (8 August 2026). `README.md`, the `CLAUDE.md` files and `paths.py` used to refer to the formulation documents and the WSL build script as living under `Docs/`. No such folder exists — all documents are under `Docs/` at the *repository root* — and the stale references had produced two live faults, both now fixed:

- `paths.py` defined `DOCS_DIR` as `Docs/Formulation`, so `MODELIB_BUILD_SCRIPT` pointed at a non-existent file and the coupling notebook’s automatic `MoDELib3` build silently declined to run (it tests `is_file()` and returns without comment). `DOCS_DIR` is now the root `Docs/`, with `DOCS_FORMULATION` and `DOCS_MANUAL` beside it.
 - `build_modelib_wsl.sh` (now `build_modelib.sh`) derived the repository root by walking up three directories, correct only from the old location (Section 4.2).
-

1.4 Conventions used in this manual

The primary platform is Windows 11 with WSL 2 (Ubuntu): `ZrMicro` builds and runs natively on Windows, while `MoDELib3` is built and executed inside WSL, because it needs a GNU toolchain, Eigen, FFTW and SuiteSparse. Command blocks are colour-coded:

PS> a Windows PowerShell command, run from the repository root

\$ a bash command inside WSL (or any Linux shell)

On a native Linux or macOS machine the same steps apply with the obvious simplifications: there is no WSL boundary, both codes build in the same shell, the venv lives at `.DisloClusterVenv/bin/`, and `paths.py` automatically selects the native binaries.

2 Installation

2.1 Prerequisites

Table 1 lists everything the repository needs. The rest of this section walks through installing each item in order.

Table 1: Prerequisites. Windows-side tools build and run `ZrMicro`; WSL-side tools build and run `MoDELib3`.

component	version	side	purpose
Git	any recent	Windows	clone the repository
Python	3.14 (python.org)	Windows	venv, notebooks, orchestration
VS Build Tools	2022 (C++ workload)	Windows	compile <code>ZrMicro</code>
CMake	≥ 3.16	Windows	configure <code>ZrMicro</code> and <code>SUNDIALS</code>
<code>SUNDIALS</code>	7.1.1	Windows	<code>CVODE/ARKODE</code> for <code>ZrMicro</code>
WSL 2 + Ubuntu	22.04 or 24.04	WSL	build and run <code>MoDELib3</code>
apt packages	Sec. 4.2	WSL	installed by the build script

Warning. Do not use **Anaconda Python** for this project. Its NumPy 1.x/2.x mix conflicts with the SciPy pinned in `requirements.txt`. Install CPython 3.14 from <https://www.python.org> and build the venv from that interpreter.

2.2 Step 1: Windows tools

1. **Git for Windows** — <https://git-scm.com>. Default options are fine.
2. **Python 3.14** — <https://www.python.org/downloads/>. Install for all users (the manual assumes `C:\Python314\python.exe`; adjust paths if you install elsewhere). It is not necessary to add it to `PATH` — every command in this manual calls an interpreter by explicit path.
3. **Visual Studio 2022 Build Tools** — <https://visualstudio.microsoft.com/downloads/>, workload “Desktop development with C++”. This provides the MSVC compiler and the OpenMP runtime that ZrMicro’s parallel batch mode uses.
4. **CMake** — <https://cmake.org/download/>, and let the installer add it to `PATH`. Any CMake ≥ 3.16 works for the Windows-side builds.

2.3 Step 2: clone the repository

From the directory that should contain the repository (any location works; no path is hard-coded):

```
PS> git clone https://github.com/Ghoniem/DisloCluster.git
PS> cd DisloCluster
```

The repository is private, so the clone authenticates with your GitHub account (Git for Windows will pop up a browser-based credential prompt the first time).

All subsequent commands in this manual are issued *from the repository root* unless stated otherwise.

Note. DisloCluster was assembled from two previously separate repositories, which were submodules until commit 859af47 converted them to plain directories. It is now a single git checkout. The root marker file `.dislocluster_root` — not any `.git` — is what identifies the repository root to `paths.py`, and `paths.git_hash()` still falls back through `root` $\rightarrow$ `ZrMicro` $\rightarrow$ `MoDELib3` so that a split checkout would keep producing meaningful provenance tags.

2.4 Step 3: WSL and Ubuntu

MoDELib3 needs a Linux toolchain. On Windows this is provided by WSL 2:

```
PS> wsl --install
```

Reboot when prompted, then complete the Ubuntu first-run setup (create a Linux username and password). Verify:

```
PS> wsl -e bash -c "lsb_release -d && gcc --version | head -1"
```

The compilers and libraries MoDELib3 needs inside WSL are installed automatically by the build script in Section 4.2; nothing further is required here.

2.5 Step 4: SUNDIALS 7.1.1

ZrMicro's C++ solver links against SUNDIALS 7.1.1 (modules `cvode`, `arkode`, `nvecserial`, `sunlinsolband`, `sunlinsolspgmr`). The ZrMicro CMake build looks for it in two places, in order: `<repo>/Libraries/sundials-7.1.1/` (preferred), then the system-wide CMake package registry (e.g. an install under `C:\Program Files (x86)\SUNDIALS`).

To build and install it into the repository tree (recommended — it keeps the whole project self-contained), run from a scratch directory:

```
PS> curl.exe -LO https://github.com/LLNL/sundials/releases/download/v7.1.1/sundials-7.1.1.tar.gz
PS> tar -xzf sundials-7.1.1.tar.gz
PS> cmake -S sundials-7.1.1 -B sundials-build '
    -DCMAKE_INSTALL_PREFIX=<path-to>\DisloCluster\Libraries\sundials-7.1.1" '
    -DBUILD_SHARED_LIBS=OFF -DBUILD_STATIC_LIBS=ON '
    -DEXAMPLES_ENABLE_C=OFF -DEXAMPLES_INSTALL=OFF
PS> cmake --build sundials-build --config Release --target install
```

Replace `<path-to>` with the actual location of your clone. The install step populates `Libraries/sundials-7.1.1/` with `include/`, `lib/` and the CMake package files that `find_package(SUNDIALS)` consumes.

3 The Python virtual environment

3.1 Create the venv and install dependencies

The whole repository shares one virtual environment, `.DisloClusterVenv/`, at the repository root. From the repository root:

```
PS> C:\Python314\python.exe -m venv .DisloClusterVenv
PS> .DisloClusterVenv\Scripts\python.exe -m pip install -r requirements.txt
```

`requirements.txt` pins the full stack — NumPy 2.4, SciPy 1.17, pandas, Matplotlib, openpyxl (the Excel workbook reader), the Gmsh SDK (mesh generation for the Adaptive notebook), and the complete JupyterLab toolchain — so the install is reproducible on any machine.

3.2 Register the Jupyter kernel

The notebooks expect a kernel named `dislocluster`:

```
PS> .DisloClusterVenv\Scripts\python.exe -m ipykernel install --user '
    --name dislocluster --display-name "Python 3.14 (DisloCluster)"
```

When a notebook is opened in JupyterLab or VS Code, select the kernel “Python 3.14 (DisloCluster)”.

3.3 Verify path resolution

Before building anything, confirm that the path-resolution layer sees your checkout correctly:

```
PS> .DisloClusterVenv\Scripts\python.exe dislocluster_code\paths.py
```

This prints one OK/MISS line per resolved location (repository root, venv, 0-D code, solver binary, MoDELib3 root and build, material file, coupled simulation case, ...). At this stage MISS is expected for the two solver binaries — they have not been built yet — and everything else should

read OK. Re-run this command whenever something cannot be found; it is the fastest diagnostic in the repository.

4 Building the two codes

4.1 The 0-D ZrMicro solver (Windows, native)

From the repository root:

```
PS> cmake -S ZrMicro\cpp_utils -B ZrMicro\build -DCMAKE_BUILD_TYPE=Release
PS> cmake --build ZrMicro\build --config Release
```

This produces `ZrMicro\build\Release\solver.exe`. Use `Release`: the parameter-identification utilities call the solver thousands of times, and the OpenMP batch mode — which the Adaptive march relies on — is only worth having when optimized. OpenMP is detected automatically by CMake; the solver still builds and runs serially without it, and the configure output says which happened (OpenMP enabled for solver (parallel batch mode) or OpenMP NOT found).

If CMake cannot find SUNDIALS, check that the install prefix of Section 2.5 is exactly `(repo)/Libraries/sundials-7.1.1/`; that path is the first entry on the build's `CMAKE_PREFIX_PATH`.

4.2 The 3-D MoDELib3 code

MoDELib3 is built by the script `Docs/Formulation/build_modelib.sh`, which installs the dependencies its platform needs, discards any build cache configured under a different absolute path (which is what happens when a checkout is moved), and builds the headless targets. The same script serves Linux, macOS and WSL; only the launcher differs. From the repository root:

```
PS> wsl -u root -e bash Docs/Formulation/build_modelib.sh MoDELib3
```

```
$ bash Docs/Formulation/build_modelib.sh MoDELib3 # Linux, macOS, WSL shell
```

Note. Fixed (8 August 2026). The script's no-argument default used to walk up *three* directories from its own location, which was correct only while it sat at `Docs/Formulation/`. From `Docs/Formulation/` — two levels below the root — that resolved one directory *above* the checkout, and the script stopped with “CMakeLists.txt not found”. Only the explicit-argument path, which the coupling notebook uses, still worked. It now walks up two levels, so the no-argument form works again:

```
PS> wsl -u root -e bash Docs/Formulation/build_modelib.sh
```

Passing `MoDELib3` explicitly remains unambiguous from any location. (Running as `-u root` avoids an interactive `sudo` password prompt for the package installs.)

The script performs, in order:

1. Dependency installation through whichever package manager the platform has — `apt-get`, `dnf` or `pacman` on Linux, Homebrew on macOS — covering the toolchain, `cmake`, `ninja`, Eigen, OpenMP, FFTW, SuiteSparse, Boost and `pybind11`. Set `SKIP_DEPS=1` to skip this step when the libraries are already in place. Only a C++20 compiler, CMake ≥ 3.16 and Eigen 3 are required; everything else is optional and its absence is reported in the configuration summary rather than fatal.

2. Stale-cache detection: if `build/CMakeCache.txt` was configured for a different source path (a moved or copied checkout), the build tree is removed and reconfigured cleanly.
3. Configure and build: `Release`, `Ninja` when it is installed and the default generator otherwise, with `-j` set from the core count (`nproc` on Linux, `sysctl` on macOS).

Note. Changed. The script was `build_modelib_wsl.sh` and, on every run, `sed`-patched the inherited `CMakeLists.txt`: rewriting the Eigen include path to Debian’s `/usr/include/eigen3`, deleting the macOS Qt and MacPorts RPATH lines, and commenting `add_subdirectory(DDqt)` out of `tools/CMakeLists.txt`. The source tree therefore carried whichever platform’s edits had been applied last, and the committed `CMakeLists` could only build on Linux. The CMake files now make those decisions themselves — each dependency is searched for, each optimization flag is probed against the compiler in hand, `DDqt` is built when Qt6 and VTK are found — so one script and one `CMakeLists` serve Linux, WSL and macOS alike. The `-DCMAKE_POLICY_VERSION_MINIMUM=3.5` workaround is gone with it: `cmake_minimum_required` is now 3.16, which CMake 4 accepts directly.

The build takes 10–30 minutes from clean. Its artifacts are `MoDELib3/build/tools/DDomp/DDomp` (a native executable — invoked directly on Linux and macOS, and from Windows through `wsl.exe`, which `paths.ddomp_cmd()` decides per platform) and `MoDELib3/build/tools/pyMoDELib/pyMoDELib*.so` (optional Python bindings, built only when `pybind11` is installed).

4.3 Verify the builds

Re-run the resolution report of Section 3.3:

```
PS> .DisloClusterVenv\Scripts\python.exe dislocluster_code\paths.py
```

The rows `0-D solver` and `MoDELib DDomp` should now read `OK`. If either is `MISS`, consult Section 9.

Warning. The governing rule of this stack: almost every failure exits zero. The `MoDELib3` build script’s final artifact check, `DDomp` with a missing initial configuration, a missing material key, a CRLF-damaged mesh, and several solver failures all return status 0. Verify by artifacts on disk and by log tails, never by exit status. The `OK/MISS` report above exists for exactly this reason.

5 First simulation: the 3-D single-crystal test

5.1 The case

`MoDELib3/tutorials/zrmicro_coupled/` is the standalone 3-D verification case, independent of the Adaptive march: a single cubic crystal of edge $1\ \mu\text{m}$ ($3093\ b$ per side, 24 115 finite-element nodes for the cluster-dynamics trial functions) with Dirichlet conditions on *all six faces*, so the entire surface acts as a grain boundary. The irradiation conditions are $T = 573\ \text{K}$, $G = 10^{-7}\ \text{dpa/s}$, zero applied stress; the run takes 31 output steps of 1 dpa each. Loop nucleation is by cascades plus homogeneous SIA clustering; the mobile species are pinned to their equilibrium values on the boundary.

Doing this case first exercises, in one pass, everything built in Sections 3 and 4: the `venv`, the path resolution, the WSL bridge, `DDomp`, and the Python post-processing.

5.2 Quick start: figures in minutes

The checkout ships with the `ev1/` output of a completed 31-dpa solve, so post-processing can be run *immediately* — no solve required:

```
PS> .DisloClusterVenv\Scripts\python.exe '
    dislocluster_code\studies\run_zr3d_singlecrystal.py '
    --doses 1 5 10 30 --max-nm 150
```

This reads `ev1/` only — it never runs a solve — and writes a timestamped run directory `Simulations/output/<stamp>_<hash>_zr3d_ghoniem/` containing the figure sets of Table 2. The command-line options are `-doses` (which snapshots to render), `-max-nm` (depth of the grain-boundary profiles), `-tag` (suffix for the run directory), `-sim-dir` (post-process a different simulation directory), and `-no-overlays` (skip the loop-platelet overlays, which dominate the runtime).

Table 2: Output of the single-crystal post-processing.

path	contents
<code>3d/mobile_fields.png</code>	C_v , C_i , C_{2i} , C_{3i} on an interior mid-plane cut, one column per dose
<code>3d/immobile_density_fields.png</code> , <code>3d/immobile_content_fields.png</code>	the four loop families, same layout
<code>3d/loops_{c,a1,a2,a3}_overlay.png</code>	one family per figure: content field with loop platelets drawn at the true local radius, exaggerated $\times 3$ ($\langle c \rangle$) or $\times 14$ ($\langle a \rangle$)
<code>gb/gb_{density,content,size, mobile}_profiles.png</code>	radial averages over the first 150 nm from the boundary
<code>provenance.md</code>	interior state table, size-vs-dose table, and the exact command to reproduce the run

5.3 Running the solve

To regenerate `ev1/` from scratch — or to run the case at different conditions after editing the inputs (Section 8) — run the driver script inside WSL. From the repository root:

```
PS> wsl -e bash MoDELib3/tutorials/zrmicro_coupled/clean_run.sh
```

A 31-step run to 30 dpa takes about one hour on 24 cores. The script derives every path from its own location, so it works wherever the repository sits. Each invocation:

1. refreshes `inputFiles/Zr3d_ghoniem.txt` and `inputFiles/unitCube_15K.msh` from `MoDELib3/Library/`, converting line endings to LF (a CRLF mesh makes the parser return *zero nodes with no error*; a stale material file has killed runs on a missing key — hence the unconditional refresh);
2. removes and recreates `ev1/` and `F/`, then regenerates the initial configuration `ev1/ev1_0.txt` with `microstructureGenerator` (that file is both the initial configuration and the `runID-0` output, so a bare re-run would otherwise restart from the previous run’s accumulated dose);
3. runs `DDomp`, logging to `gen.log` and `run.log` in the case directory.

Progress can be watched from another terminal with

```
PS> wsl -e bash -c "tail -f MoDELib3/tutorials/zrmicro_coupled/run.log"
```

When the run finishes, `ev1/` holds `cdNodes.txt` (the node coordinates) and one `ev1_N.txt` per output step, and the post-processing of Section 5.2 applies unchanged.

Note. Output is written *after* each solve step, so `ev1_N` holds the state after $N+1$ dose steps: `ev1_0.txt` is 1 dpa, not 0 dpa. The dose labels passed to `-doses` account for this convention automatically.

5.4 Reading the figures critically

Two properties of the output should be kept in mind when interpreting it:

- **Platelet radii in the overlay figures are exaggerated**, and the factor differs between the $\langle c \rangle$ ($\times 3$) and $\langle a \rangle$ ($\times 14$) figures — sizes are faithful *within* a figure but not *between* figures.
- **Near-boundary loop densities are not quantitative.** The Dirichlet faces are a sink for the mobile defects but not for the loops: cascade nucleation is spatially uniform while the only loop-removal channel (coalescence) is driven by the absorbed mobile flux, which vanishes where the mobile concentrations are pinned. The $\langle a \rangle$ density therefore climbs steeply toward the boundary and grows linearly in dose there. The interior of the domain — which is what is compared against the 0-D model — is unaffected.

6 The 0-D ZrMicro simulation

6.1 Running it

Open `ZrMicro/code/ZrMicro.ipynb` in JupyterLab or VS Code, select the kernel “Python 3.14 (DisloCluster)”, and run all cells. The full integration and figure suite takes minutes. Headless execution is a one-liner:

```
PS> .DisloClusterVenv\Scripts\python.exe -m nbconvert --to notebook --execute '
--ExecutePreprocessor.kernel_name=dislocluster '
--output-dir Simulations\output '
ZrMicro\code\ZrMicro.ipynb
```

The notebook builds the model chain (`InputData` $\rightarrow$ `ReactionRates` $\rightarrow$ `RateEquations`), flattens the parameter set into the `-key=value` command line that `solver.exe` consumes, integrates the 19-equation system with CVODE (BDF, dense — the recommended configuration, see Section 6.4), and writes the figure suite plus `provenance.md` to a timestamped output directory.

6.2 Inputs: the Excel workbook

All 0-D physical input lives in `Simulations/input/Zr_input_parameters.xlsx`. Three sheets are read by the solver:

- `Material_Environment` — irradiation conditions and sink microstructure: temperature T , dose rate G , applied and hydrostatic stress, network dislocation density, grain diameter, precipitate population.

- **Physical_Properties** — lattice constants, Burgers vectors, atomic volume, formation/migration/binding energies, attempt frequencies, elastic constants, cascade efficiencies.
- **Model_Parameters** — bias factors, DAD anisotropy parameters, coalescence coefficients, nucleation sizes, dissolution parameters, obstacle strengths.

The remaining sheets hold digitized experimental data (Northwood, Gilbert, Jostsons, Cockram, Tournadre, Harte and others) used only by the offline fitting utilities (`disloccluster_code/fitting/`), never by the solver.

Fitted parameters are applied *on top of* the workbook through the `PARAMETER_OVERRIDES` dictionary in the notebooks; the override always wins, and each override is routed automatically to whichever sheet already defines the key. The overrides are pinned at full precision in the notebook rather than inherited from the workbook, so that reference runs remain reproducible even as the workbook evolves (Section 10).

6.3 The state vector

The integrated state is a flat array of length 19, and the same layout is used by the Python layer, the C++ solver, and the Adaptive march:

Table 3: ZrMicro state vector.

slice	contents	meaning	units
y[0:4]	C_v, C_i, C_{2i}, C_{3i}	mobile species	atom fraction
y[4:8]	$C_{iL}, C_{aiL}, C_{vL}, C_{avL}$	loop number densities	per atom
y[8:12]	$C_{iL}^i, C_{aiL}^i, C_{vL}^v, C_{avL}^v$	loop contents	per atom
y[12:18]	six accumulators	conservation diagnostics	per atom
y[18]	ρ_N	evolving network density	m^{-2}

The four loop families are the aligned/non-aligned split of the two loop types: iL and aiL are prismatic $\langle a \rangle$ interstitial loops, vL and avL are basal $\langle c \rangle$ vacancy loops. At zero applied stress the three prismatic variants are equivalent and the aligned fraction is 1/3.

6.4 Solver configuration

The ODE backend is selected by the `solver_method` entry of the notebook’s solver configuration block:

```
'solver_method': {'backend': 'cvmde', 'lmm': 'bdf', 'linsol': 'dense'}
```

CVODE BDF with a dense linear solver is the recommended default. The accuracy benchmark against a tight-tolerance reference (at `rtol`= 10^{-6} , `atol`= 10^{-20}) is summarized in Table 4; ARKODE tables designed as IMEX pairs perform poorly in pure implicit mode on this system and should be avoided.

7 The coupled 0-D $\leftrightarrow$ 3-D simulation

7.1 The idea

The Adaptive march realizes the operator-split QSSA over each dose step $[\gamma_n, \gamma_{n+1}]$ in two alternating stages:

Table 4: ODE solver benchmark on the 0-D system.

config	backend	linear solver	max. rel. error
A (default)	CVODE BDF	dense	3.0×10^{-4}
B	CVODE BDF	band	3.0×10^{-4}
C	CVODE BDF	GMRES	1.7×10^{-2}
G	ARKODE SDIRK_5_3_4	dense	3.0×10^{-1}
F, H, I	ARKODE (other tables)	dense	10^4 – 10^7 (fails)

1. **Fast solve** — the steady mobile field $C_M^*(x)$ is computed with the immobile (loop) state frozen, either by the real MoDELlib3 finite-element solve or by an analytic placeholder (Section 7.6).
2. **Slow march** — the immobile ODEs are integrated independently at every quadrature point with the mobile species frozen (`freeze_mobile=1`), one OpenMP batch case per point, in a single `solver.exe` subprocess.

The loop state sets the sink field the mobile solve sees; the resulting mobile field then drives loop growth over the step. The splitting error is first order in $\Delta\gamma$. The state exchanged between the codes, and the modules that carry it, are listed in Table 5.

Table 5: The coupling contract.

direction	carrier	module
0-D $\rightarrow$ 3-D	loop density/radius per family $\rightarrow$ sink field	<code>modelib_fem.write_sink_field</code>
3-D $\rightarrow$ 0-D	steady mobile field C_M^* at quadrature points	<code>modelib_fem.solve</code>
0-D $\rightarrow$ 3-D	dose-indexed closure table	<code>modelib_export</code>
march	immobile ODE step, mobile frozen	<code>modelib_coupling.run_immobile_step</code>
both	material constants	<code>Library/Materials/Zr3d_ghoniem.txt</code>

For a gentle first contact with the coupling machinery, `code/coupling_demo.ipynb` is a minimal walk-through of one fast solve and one immobile step; the full driver described next is `code/coupled_0d_3d_ZrMicro.ipynb`.

7.2 Two options for the slow step

The fast solve is the same whichever way you run it: MoDELlib3’s `solveMobileClusters` has no time derivative at all, so it always returns the steady $C_M^*(x)$ for the immobile state it is handed. What you choose is how the *slow* step is integrated.

Choose **IMEX** when you want one self-contained MoDELlib3 run and the dose grid is uniform; choose **Adaptive** when the dose grid spans decades, when you need a requested accuracy rather than an assumed one, or when the microstructure is evolving fast enough that a first-order step would have to be made small everywhere to stay accurate anywhere.

7.2.0.1 The names. These were called *standalone* and *coupled* in earlier revisions and in output written before this manual. Those names described how each was driven rather than what it is, and “coupled” was misleading, since both are coupled 0-D $\leftrightarrow$ 3-D in the sense that matters. The

Table 6: The two time-integration options. Both perform the same operator split with the same fast step.

	name	slow step
Option A	IMEX	Implicit-explicit Euler with Patankar stabilization — $n \leftarrow (n + \Delta t \dot{n}_{\text{prod}})/(1 + \Delta t \lambda_{\text{loss}})$, production explicit and destruction implicit. Unconditionally positive at any step size, but first order and with no error estimate, so accuracy is set by the step size alone (20 substeps per 1 dpa output step). Runs entirely inside DDomp.
Option B	Adaptive	SUNDIALS CVODE, adaptive-order adaptive-step BDF, at every node with the mobile species frozen. Order and step size both follow a local error estimate against <code>rtol/atol</code> , so accuracy is requested rather than inferred. Driven from Python by <code>dislocluster_code/coupling/seeded_march.py</code> .

word is kept for the *architecture* — the field handoff of Table 5 — and retired as a route name. The canonical spellings are in `dislocluster_code/integration.py`, which every driver imports so that directory names, table columns and plot legends cannot drift apart.

Note that “standalone” survives elsewhere in this manual in an unrelated sense: the *standalone 3-D single-crystal test* of Section 5 means a MoDELib3 run that is not driven by the 0-D notebook, which is a statement about who launches it rather than about which integrator it uses. That case does in fact use Option A, since a MoDELib3 run on its own has no other choice.

7.3 Starting from pristine material

The march does *not* need a 0-D seed. It can start from unirradiated material and nucleate its own loop population.

This works because the two transients are about a hundred times apart in dose. Taking the adaptive 0-D as the reference, the mobile species are settled by 10^{-4} dpa (C_i by 6×10^{-10} , C_{3i} by 1.6×10^{-8} , C_v by 4.2×10^{-5}), while loop nucleation runs from 10^{-3} to 10^{-2} dpa. That separation is exactly what the operator split assumes, so a smaller starting dose sits *further inside* the split’s domain of validity, not outside it.

Two further facts make the seed dispensable rather than merely tolerable. The seed’s mobile half was never used — the first fast solve overwrites it — so only the immobile half ever mattered; and the loop-free limit is not singular, because the dislocation network rather than the loop population carries most of the sink strength (deleting every loop raises C_v by 11%).

Measured on the 1 μm case, a march from pristine material reproduces the 0-D through the nucleation transient to within 15%, and from 1 dpa onward is indistinguishable from a march that was handed a 0-D seed — to five parts in ten thousand at 6 dpa. Full numbers are in `Docs/DisloCluster Manual/solver_and_coupling_rearchitecture.pdf`, Section “Removing the 0-D seed entirely”.

```
# stage a fresh domain and let MoDELib write its own CD node set
python -m dislocluster_code.staging.domain --size-nm 500 --layer-nm 150
```

```

--layers-across 5 --interior-lc-nm 120 --name zr3d_500nm

# march it from pristine material on a logarithmic dose grid
python -m dislocluster_code.coupling.seeded_march --doses 0 1e-3 1e-2 0.1 1 6
--substeps 1 --fem-every 2 --tag Adaptive_pristine
--template ../../MoDELib3/tutorials/zr3d_500nm

```

(each command is one line; it is wrapped here only to fit the page.)

A seed dose of 0 selects pristine material. Seeding from the 0-D remains convenient when the interest is at doses well past the transient, since it skips three decades of dose for a handful of fast solves — but it is a convenience, not a requirement, and it is the weaker choice whenever the transient itself matters, because a 0-D seed imposes a spatially uniform loop population that the 3-D would never produce.

7.4 The notebook’s three control blocks

The coupled notebook is driven entirely from three dictionaries near its top; nothing else in the notebook needs editing for a normal run.

Table 7: Control blocks of `coupled_0d_3d_ZrMicro.ipynb`.

cell	dictionary	controls
6a	GEOMETRY, MESH	domain type (<code>cubic</code> / <code>hexagonal</code>), dimensions, element size and order, boundary-layer refinement
6b	RUN	temperature, dose rate, stress, dose schedule, tolerances, substeps, march grid
6c	PARAMETER_OVERRIDES	0-D parameters applied on top of the Excel workbook

The dose schedule is `n_intervals` equal steps from `dose_seed` to `dose_max`; the defaults (1 → 26 dpa in 5 intervals) give snapshots at 1, 6, 11, 16, 21 and 26 dpa. The immobile substep count is fixed *per dose interval* by `substeps_per_interval` rather than by a wall-clock cap — a 5 dpa step at 10^{-7} dpa/s spans 5×10^7 s, which a fixed time cap would have split into a thousand batch subprocesses.

7.5 Meshing, and the materialized simulation directory

Cell 6a writes the finite-element mesh through `Gmsh/generate_mesh.py` (cubic or hexagonal domains, optional boundary-layer refinement); generated `.msh` files are cached under `Gmsh/meshes/` and are reproducible from the spec, so they are not committed.

After the run directory is created, the notebook materializes a complete, self-contained MoDELib3 simulation directory at `<run>/sim/`: the generated mesh, a deformation gradient `F` restoring the physical domain size, and a `DD.txt` whose dose schedule matches the notebook’s snapshots. **The 3-D solve itself is not launched** — the notebook prints the exact `ws1` command to run it, and the figure cells render from whichever output directory `EVL_DIR` points at (by default the shipped verification case of Section 5, so the figure cells always have data).

7.6 Fast-solve backends

The fast mobile solve inside the march is selected by `FAST_SOLVE_BACKEND`:

Table 8: Fast-solve backends.

value	behaviour
auto (default)	MoDELib3 FEM if DDomp is built <i>and</i> the simulation directory is seeded, else the placeholder
modelib	require the real FEM solve; raise if unavailable
placeholder	analytic $C_M^*(x) = [1 - e^{-x/\ell}] C_M^{\text{bulk}}$ — pipeline checks only

Warning. The placeholder is *not* a MoDELib3 result. It scales all four mobile species by the same factor, which cannot represent species-dependent depletion lengths that differ by four orders of magnitude; its boundary profile is an artifact and must not be read physically. The notebook states loudly in its output which backend is active — read that message every time. To activate the real backend, seed a simulation directory once:

```
from dislocluster_code.legacy.modelib_fem import seed_sim_dir_from_tutorial
from dislocluster_code import paths
seed_sim_dir_from_tutorial(paths.MODELIB_ROOT, paths.OUTPUT_DIR / "modelib_sim")
```

then run its `generateInputFiles.py` with MoDELib3’s MicrostructureGenerator. Until that is done, `auto` silently falls back to the placeholder.

7.7 Running the march

Interactively: open `coupled_0d_3d_ZrMicro.ipynb` on the “Python 3.14 (DisloCluster)” kernel and run all cells. Headless:

```
PS> .DisloClusterVenv\Scripts\python.exe -m nbconvert --to notebook --execute '
--ExecutePreprocessor.kernel_name=dislocluster '
--output-dir Simulations\output '
ZrMicro\code\coupled_0d_3d_ZrMicro.ipynb
```

Before the march begins, the notebook (i) ensures both builds — `ensure_zrmicro_solver()` configures and builds `solver.exe` through CMake if it is absent, and `ensure_modelib()` checks for DDomp, invoking the WSL build script if necessary; both verify artifacts on disk rather than trusting exit status — (ii) runs the full 0-D reference integration, which supplies the physical reference and the initial immobile state, and (iii) performs a field-by-field consistency check between the MoDELib3 material file and the 0-D conditions, reporting both the matches and the *known* non-correspondences (Appendix B) so that a residual is not mistaken for a bug.

7.8 Outputs

Every coupled run writes `output/<YYYYMMDD_HHMMSS>_<git-hash>_coupled0d3d/` containing `march_state.npz` (the full march state), `provenance.md`, and three figure directories:

Two interpretation caveats, in addition to those of Section 5: the loop-overlay figures here fill the domain with non-overlapping platelets at radii exaggerated $\times 1.5$ ($\langle c \rangle$) and $\times 7$ ($\langle a \rangle$), and they are the slow figures (~ 40 s each). The `size_dist_*` figures show the distribution of the *local mean* loop diameter across the domain, weighted by local loop density — not a per-loop size spectrum; the model carries one mean size per family per node, so spatial variation is the only source of spread.

Table 9: Coupled-run figure output.

directory	contents
0d/	the 0-D figure suite, unchanged
3d/	fe_mesh.png, plus one file per quantity per dose: Cv_06dpa.png, N_c_06dpa.png, C_a1_06dpa.png, loops_c_06dpa.png, ...
gb/	one file per quantity with all doses overlaid (gb_Cv.png, gb_N_a1.png, gb_C_c.png, gb_d_c.png), plus size_dist_{family}_{dose}.png

8 Changing things

Table 10 maps the common modifications onto the files and control blocks that own them.

Table 10: Where to change what.

to change	edit
irradiation conditions (T, G, σ)	the RUN dictionary of the Adaptive notebook <i>and</i> inputFiles/polycrystal.txt / Zr3d_ghoniem.txt for the 3-D case; they are not linked
0-D material parameters	the workbook input/Zr_input_parameters.xlsx, or PARAMETER_OVERRIDES (which wins)
3-D material parameters	MoDELib3/Library/Materials/Zr3d_ghoniem.txt, then re-run clean_run.sh so the copy in inputFiles/ is refreshed
dose schedule (Adaptive march)	dose_seed, dose_max, n_intervals, substeps_per_interval in RUN
dose schedule (3-D single crystal)	Nsteps and dtMax in tutorials/zrmicro_coupled/inputFiles/DD.txt
mesh or domain (Adaptive march)	GEOMETRY / MESH in cell 6a (regenerates through Gmsh)
mesh or domain (3-D single crystal)	meshFile and the deformation gradient F in inputFiles/polycrystal.txt
0-D solver method	solver_method in the solver configuration (Section 6.4)

Three of these deserve elaboration.

8.0.0.1 Temperature appears in three places. The workbook, the notebook’s RUN block, and polycrystal.txt each carry a temperature, and they are not linked. Moreover the DAD reconciliation parameters dadAnisotropy (p_m) and dadZ0 (Z_m^0) in Zr3d_ghoniem.txt are *fitted at* $T = 573$ K by closed-form inversion against the 0-D symmetric capture efficiencies. Changing the temperature requires refitting them, or the DAD bias will be silently wrong.

8.0.0.2 The 3-D dose step is expressed in MoDELib3 time units. In DD.txt, dtMax = 6.95869×10^{19} corresponds to 1 dpa at $G = 10^{-7}$ dpa/s (the MoDELib3 time unit is $b/c_s = 1.437 \times 10^{-13}$ s; see the unit table in Appendix A). Scale it proportionally when changing the dose increment or the dose rate.

8.0.0.3 Keep startAtTimeStep at zero. Never set it to -1 : that silently restarts from a stale ev1_N and double-counts dose. clean_run.sh pins it and regenerates ev1_0.txt for exactly this reason.

9 Troubleshooting

The single most important habit was already stated in Section 4: *almost every failure in this stack exits zero*, so verify by artifacts on disk (the OK/MISS report, log tails, file sizes) rather than by exit status. Table 11 collects the known symptoms.

Table 11: Symptoms and causes.

symptom	cause and fix
<code>paths.py</code> reports MISS for a location that exists	The checkout deviates from the standard layout. Set <code>DISLOCLUSTER_ROOT</code> , <code>MODELIB_ROOT</code> and/or <code>MODELIB_BUILD</code> explicitly and re-run the report.
<code>pip install</code> fails or SciPy import errors	Anaconda Python was used to create the venv. Recreate it from the python.org CPython 3.14 (Section 3).
CMake cannot find SUNDIALS	The install prefix is not <code>(repo)/Libraries/sundials-7.1.1/</code> and no system install is registered. Redo Section 2.5.
MoDELib3 CMake <i>generate</i> step fails after a successful configure	<code>libfftw3-dev</code> is missing — the library is linked unconditionally despite the “noise generator disabled” message. The build script installs it; if building by hand, install it first.
MoDELib3 reconfigure hard-errors: “the current CMakeCache.txt directory ... is different”	The checkout was moved or copied, so the cached absolute paths are stale. <code>build_modelib.sh</code> detects and discards the stale cache; by hand, delete the <code>build/</code> tree and re-configure. The same applies to <code>ZrMicro/build/</code> on the 0-D side.
<code>build_modelib.sh</code> stops with “CMakeLists.txt not found”	The script was run without the explicit MoDELib3 argument (Section 4.2).
<code>microstructureGenerator</code> never terminates	It is running the discrete-dislocation path, inserting loops one at a time; continuum loop densities would need $\sim 10^{11}$ of them. Set <code>useDislocations=0</code> and zero target densities — the shipped <code>inputFiles/</code> already do.
Parser reports zero mesh nodes, no error	CRLF line endings in the <code>.msh</code> . <code>clean_run.sh</code> LF-converts it on every run for this reason.
Run restarts from the previous dose	<code>ev1_0.txt</code> is both the initial configuration and the <code>runID-0</code> output. Regenerate it every run (<code>clean_run.sh</code> does).
Loop content comes out at exactly $3\times$ the expected value	<code>startAtTimeStep=-1</code> resumed from a stale <code>ev1_N</code> and double-counted dose. Pin <code>startAtTimeStep=0</code> .
Newton residual stalls near 10^5	An initial-guess problem, not a linear-algebra one; the iterative (BiCGSTAB) branch is the working one on the $\sim 96k$ -dof system.
Notebook silently produces analytic mobile profiles	The fast-solve backend fell back to <code>placeholder</code> . Check that <code>DDomp</code> is built and the simulation directory is seeded (Section 7.6).
C_{2i} , C_{3i} orders of magnitude low	Historically a member-initialization-order bug in <code>ClusterDynamicsParameters</code> (fixed on this branch; see Appendix B). If it reappears, check that no member’s initializer reads a member declared after it.

symptom	cause and fix
Three $\langle a \rangle$ families give different densities at zero stress	Burgers vectors supplied in Cartesian rather than lattice components. The zero-stress symmetry check is the fastest detector of this error class.
Notebook regression cell reports “skipped”	The pinned 0-D baseline run directory is not present in this checkout. Point <code>REFERENCE_RUN</code> at any local run directory to re-enable the regression.

10 Reproducibility and provenance

Every run — 0-D, 3-D post-processing, or coupled — writes a timestamped, git-tagged directory

`Simulations/output/(YYYYMMDD_HHMMSS)_{git-hash}[_{tag}]/`

containing the figures and `provenance.md`, which records the resolved repository layout, every input parameter, the active fast-solve backend (for coupled runs), the software environment, and the solver statistics — so every figure and every number is traceable to a commit. `paths.git_hash()` falls back through `root` → `ZrMicro` → `MoDELib3`, so the tag stays meaningful even in a split checkout.

Warning. `provenance.md` stores values to four significant figures only, so re-reading it will *not* reproduce a run bit-for-bit. The full-precision fitted parameter set lives in the `PARAMETER_OVERRIDES` block of the notebooks, which is why that block is pinned rather than inherited from the workbook.

A Architecture reference

This appendix condenses the full architecture description (the companion document *Zr3d Code Architecture*, `Docs/Formulation/`) to what a user needs when reading code or diagnosing a run.

A.1 The governing split

The mobile species are posed as a quasi-steady reaction–diffusion boundary-value problem with no time derivative,

$$0 = -\nabla \cdot \mathbf{J} + P_M + \frac{1}{2} \mathbf{c}_M^T R_2 \mathbf{c}_M + (E - D) \mathbf{c}_M + S_{vL}, \quad (1)$$

and the immobile loop families are marched pointwise in dose against the frozen mobile field. Both execution routes realize this split:

- **Route A — notebook-driven Adaptive march** (Section 7). Python owns the dose loop; `MoDELib3` solves the fast mobile BVP; the `ZrMicro` C++ integrator advances the immobile state with `freeze_mobile=1`, one OpenMP batch case per quadrature point.
- **Route B — monolithic MoDELib3 run** (Section 5). Everything happens inside `DDomp::ClusterDynamicsFEM::solveMobileClusters()` performs the Newton solve of Eq. (1) and `solveImmobileClusters()` marches the loop families node by node. No Python at run time.

Table 12: The components a user is most likely to touch. Route marks where each participates: A = Adaptive march, B = monolithic run, P = post-processing.

component	route	role
dislocluster_code/paths.py	A,B,P	canonical path resolution (Section 1.3)
dislocluster_code/zerod/input_data.py	A	reads the workbook, computes every derived quantity
dislocluster_code/zerod/reaction_rates.py, rate_equations.py	A	the 0-D physics; deliberate mirror of the C++ RHS
dislocluster_code/zerod/cpp_bridge.py	A	flattens parameters to the -key=value CLI, launches solver.exe, parses stdout
dislocluster_code/coupling/immobile.py	A	the operator-split driver (pack_y0, run_immobile_step)
dislocluster_code/legacy/modelib_fem.py	A	adapter to MoDELib3: writes the sink field, invokes DDomp (through ws1.exe when required), reads the mobile field back
dislocluster_code/coupling/export.py	A	maps the lumped 0-D loop families onto the three $\langle a \rangle$ prismatic variants
dislocluster_code/post/report.py, modelib_fields.py, modelib_gb.py	B,P	read ev1/, render field panels, overlays and grain-boundary profiles
dislocluster_code/studies/run_zr3d_single_crystal.py	B,P	the single-crystal post-processing driver (Section 5)
code/solver.cpp + cpp_utils/	A	the 0-D integrator: CVODE/ARKODE, OpenMP batch mode, one SUNContext per thread
MoDELib3/tools/DDomp/	B	the 3-D runner
ClusterDynamics(FEM,Parameters)	B	the 3-D numerical core: weak forms, mobile Newton iteration, immobile march
<i>anisotropic diffusion and the discrete handoff (Section A.6)</i>		
dislocluster_code/staging/ anisotropy.py	A	generates the diffusion tensor <i>and</i> dadAnisotropy from one p_m per species, so the two cannot disagree
dislocluster_code/post/ coarsening.py	A,P	the d_{coarsen} detector: where the mean-field treatment of coalescence stops being valid
dislocluster_code/coupling/ transition.py	A	performs the continuum→discrete handoff and audits it against three invariants
dislocluster_code/coupling/ neighbors.py	A	the screening length $L_s = 1/k$ and the cutoff R_c that makes the climb solve affordable
dislocluster_code/coupling/ ellipse_rom.py	A	the prismatic loop as two degrees of freedom rather than $2N_{\text{sides}}$
dislocluster_code/studies/ dad_sweep.py, dad_window.py	P	which anisotropy admits simultaneous growth of both families

A.2 Key components

A.3 The three interfaces

All cross-component state flows through three deliberately narrow interfaces:

1. **Python** $\rightarrow$ **ZrMicro C++**: a flat `-key=value` command line, one token per scalar; results return as a whitespace-delimited matrix on stdout. In batch mode, one case per line of the batch file, each output block prefixed `=== CASE <i> status=<s> ===`. Purely textual — no build-time coupling, so either side can be rebuilt independently.
2. **Python** $\leftrightarrow$ **MoDELib3**: the file system. `write_sink_field()` writes the loop state into `inputFiles/aLoopsDensity.txt` and `inputFiles/frankLoopsDensity.txt`; DDomp runs on the directory; the mobile field is read back from its output. On Windows this crosses the WSL boundary, with Windows paths translated to `/mnt/<drive>/...`
3. **MoDELib3** $\rightarrow$ **post-processing**: `evl/evl_N.txt` (12 columns per node: four mobile species, four loop densities, four loop contents) plus `evl/cdNodes.txt` (the node coordinates). The second file is essential: the cluster-dynamics trial functions live on second-order elements, so the field rows do *not* correspond to the mesh vertices and the ordering cannot be reconstructed externally.

A.4 Unit systems

The two codes do not share a unit system, and most historical reconciliation defects were unit errors.

Table 13: Unit conventions.

quantity	ZrMicro	MoDELib3
length	m	$b = 3.233 \times 10^{-10}$ m
time	s	$b/c_s = 1.437052 \times 10^{-13}$ s
mobile concentration	atom fraction	atom fraction (same)
loop number density	per atom	per b^3
loop content	per atom	volume fraction (numerically the same)

With $\Omega_{b^3} = \Omega/b^3 = 0.355105$, the density conversion is $N_{0D} = N_{3D} \Omega_{b^3}$, and the defects per loop follow as $m_k = c_k/(N_k \Omega)$. One dose step of 1 dpa at $G = 10^{-7}$ dpa s $^{-1}$ is 6.95869×10^{19} MoDELib3 time units.

A.5 Call graph

Figure 1 is the component call graph across both routes.

A.6 Anisotropic diffusion and the discrete handoff

Two capabilities were added on 16 August 2026 and both change how a case is set up, so they belong in the architecture rather than only in the change log.

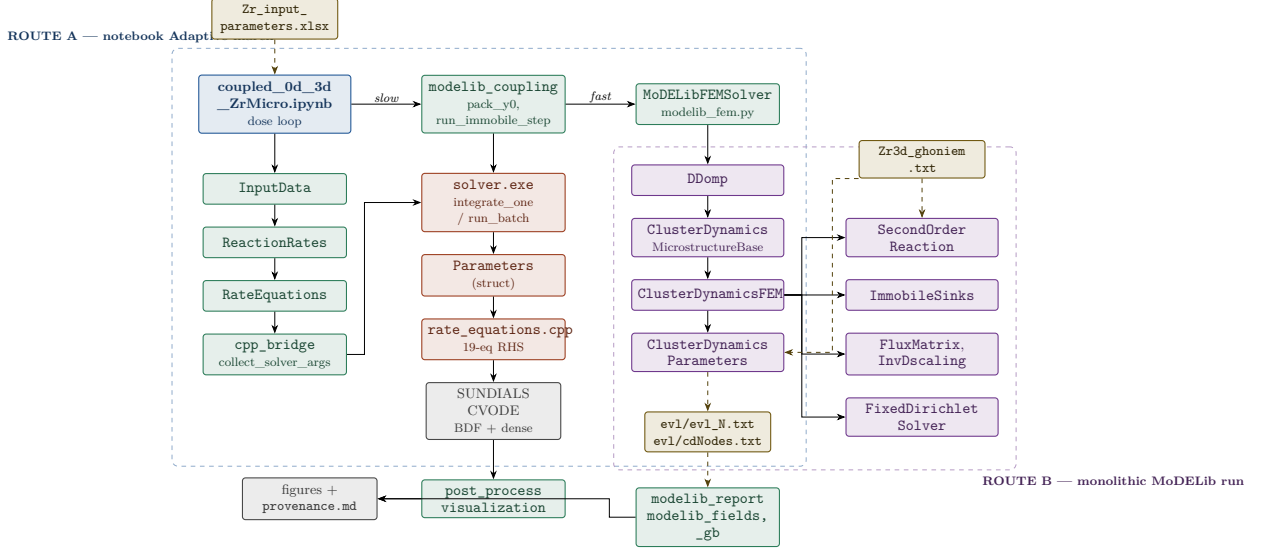


Figure 1: Component call graph. Solid arrows are calls, dashed arrows are file input/output. Blue: orchestrating notebook; green: ZrMicro Python; red: ZrMicro C++; purple: MoDELlib3 C++; ochre: files on disk; gray: products. Route A drives the fast mobile solve through MoDELlib3 and the slow immobile march through the ZrMicro integrator; Route B runs both inside MoDELlib3. The two routes share the material definitions and the post-processing layer.

A.6.0.1 One anisotropy factor, two representations. The diffusional anisotropy difference appears in the material file twice: as `mobileSpeciesEnergyMigration_eV`, the tensor the fast solve diffuses with, and as `dadAnisotropy`, the p_m that Woo’s closed-form capture efficiencies use. These used to be set independently, so the code could simultaneously believe that diffusion was isotropic and that the sinks were biased by its anisotropy. `staging/anisotropy.py` now generates both from one p_m per species, splitting the migration energy at fixed $D_{\text{eff}} = (D_a^2 D_c)^{1/3}$ so that changing p_m changes the *directionality* of diffusion and not its overall rate — which is what leaves the existing calibration intact:

$$E_m^{(11,22)} = E_m^{\text{eff}} + 2k_B T \ln p_m, \quad E_m^{(33)} = E_m^{\text{eff}} - 4k_B T \ln p_m. \quad (2)$$

Note the temperature dependence this introduces: because the anisotropy is stored as energies, $p_m(T) = \exp(-\Delta E_m / 6k_B T)$, so a pair fixed to give 0.7 at 573 K gives 0.649 at 473 K. That is physically right — anisotropy weakens as temperature rises — and it repairs a fit previously frozen at one temperature.

A.6.0.2 Which anisotropy to use. Both loop-growth conditions reduce to bounds on a single number, the arrival ratio $A/B = \bar{D}_v c_v / \sum_m \bar{D}_m c_m |m|$, and the window between those bounds is non-empty if and only if $p_I < p_v$. The measured co-growth set is

$$p_m = (p_v, p_i, p_{2i}, p_{3i}) = (1.000000, 0.913720, 0.913720, 0.913720), \quad (3)$$

vacancies isotropic and the interstitials at the value `dadAnisotropy` already carried, with tolerance $p_v \in [0.94, 1.07]$. Two limits must be quoted with any use of this: the criterion governs the *absorption flux* only — the content update also carries cascade nucleation, which no anisotropy affects — and an isotropic march is not a zero of the criterion but simply another parameter point, so a ratio taken against one cannot test a statement about signs.

A.6.0.3 The handoff. `post/coarsening.py` decides *when* the continuum description of a loop family stops being adequate, reporting d_{coarsen} where the Avrami overlap fraction crosses φ^* . `coupling/transition.py` performs the switch at the next fast-solve boundary, where a `DDomp` call happens anyway and the two sides are already synchronized. It is armed by `COUPLING['discrete_transition']` and is **off by default**. Three invariants are audited across every transfer: loop number, stored defects (exact to 10^{-16}), and sink strength — the last of which *jumps*, by design, and is reported rather than asserted on.

Two practical constraints govern whether a transfer is possible at all. `microstructureGenerator` silently refuses loops whose nodes leave the grain, so `fits_in_crystal` predicts that decision and keeps the refused share in the continuum; and the domain must be large enough to contain the loops, which is not automatic. On the 500 nm case the kept fraction is 0.997 at 0.1 dpa and 0.55 from 1 dpa onward, the latter a purely geometric plateau; on the 200 nm case it is exactly zero, because the basal radii reach 65 nm against a 173 nm extent. **Check the kept fraction before trusting a transfer on any new geometry.**

A.7 Known non-correspondences between the 0-D and 3-D models

These are physics, not bugs — do not “fix” them silently:

1. **DAD bias.** The 0-D uses phenomenological anisotropy parameters δ_i, δ_v with $Z_a + Z_c = 2$; the 3-D generates the bias from the diffusion tensors via p_m . They are reconciled by a closed-form fit of (Z_m^0, p_m) in `Zr3d_ghoniem.txt`, valid only at the fitted temperature. `Zr4.txt` is the standalone 3-D calibration and carries no `dad*` keys — do not check against it.
2. **Bi-pyramid vacancy family** — exists only in the 3-D model.
3. **Size-dependent vacancy-loop thermal emission** — 3-D only; the 0-D uses a constant surrogate.

B Changes relative to upstream MoDELib

B.0.0.1 Which upstream. The baseline is `m1m335/MoDELib2>NNL`, at the point the immobile population equations were written into it. That code solves the mobile steady-state PDE of Eq. (1) and nothing else: its immobile species count is zero, `solveImmobileClusters()` is an empty function body, and there is a literal `// Missing immobile sinks` placeholder where the loop term would go. The loop populations were *absent*, not present in a simplified form, and the list below is stated against that.

`m1m335/MoDELib-fullCD` is a *separate* cluster-dynamics branch: `iSize` is 8, it has its own immobile integrator (`ImmobileSinkRate.h`, `SpatialODESolver.h`), the loop sink on the mobile species (`FirstOrderReaction.h`), and a continuum-to-discrete loop conversion that `MoDELib3` does not have. It is a sibling, not the ancestor.

B.0.0.2 A correction. An earlier revision of this manual declared `fullCD` the correct baseline and `MoDELib3`’s immobile solver a *re-discretization* of its scheme, on the evidence that 31 cluster-dynamics parameter keys are shared verbatim and most of the rest are systematic renames. **That inference does not follow.** Shared naming across two branches of one code, developed in one group from one formulation, is evidence of *convergence* and says nothing about which came first. The immobile equations here were written to reproduce `ZrMicro`, not to re-express `fullCD`. A full numbered history is given in the companion note `DisloCluster Code History`.

The two schemes remain worth comparing as siblings, and differ in the discretization: fullCD projects its rate through the consistent mass matrix and takes one explicit-Euler step of `dtMax` followed by sequential implicit loss factors, where `MoDELiB3` evaluates the rate at nodes and takes 20 sub-steps of a single fused semi-implicit update.

Relative to `MoDELiB2-MNL` the entire immobile half of the model is new. The following are the pieces of it, with the sibling comparison noted where one exists:

- the loop density and content evolution equations (`solveImmobileClusters()`), re-discretized as above;
- homogeneous SIA-clustering nucleation, read off the reaction network the mobile solve already uses (`getLoopNucChannels()`);
- the loop absorption sink on the mobile species (`ImmobileSinks.h`), deliberately excluding the grain boundary, which the spatially resolved model captures through the Dirichlet condition and would otherwise be counted twice;
- DAD capture efficiencies from (p_m, Z_m^0) (`loopDADbias()`);
- absorption fluxes with cluster weighting, the minimum-stable-size gate, vacancy-loop dissolution, size-dependent thermal vacancy emission, and two-channel Avrami coalescence;
- a screened cutoff on the Galerkin climb pair assembly, through the optional material key `climbNeighborCutoff_b`. The assembly runs over every ordered pair of segments and is 100% of the climb-solve cost — the live path is lumped, so its “solve” is one scalar division per node. Truncation is legitimate because the kernel is not bare $1/r$ but screened as e^{-kr}/r by the same k^2 `ImmobileSinks.h` already assembles, and because the continuum field already carries the mean-field response of the whole population, so the discrete sum must supply only the near-field correction. Zero or absent means no cutoff, so every material file written before the key existed is unaffected; an enormous cutoff reproduces the uncut assembly bit-for-bit. The measured truncation error on the basal plastic-distortion rate is 19.3%, 9.6%, 3.9% and 0.25% at R_c/L_s of 1, 2, 3 and 4 respectively, so the default is 4.

Four defects were also corrected in pre-existing upstream code: a member-initialization-order bug that silently dropped the Boltzmann factor from the cluster dissociation rates; a spurious scaling of the diffusion operator relative to the reaction terms after the cluster-dynamics atomic volume became independently settable; a unit error that left the minimum-stable-radius gate permanently open; and the use of the two-atom hexagonal lattice *cell* volume where the atomic volume was intended (now settable explicitly via the optional `atomicVolume_SI` material key).

On the input side, Po’s standalone calibration `Zr4.txt` is left untouched; the coupled, reconciled definition is the separate file `Zr3d_ghoniem.txt`, which resets the mobile block (cascade fractions, migration energies, diffusivities, reaction prefactors, network sink) to correspond with the 0-D model and adds the entire immobile block (~20 new keys). The full before-and-after, key by key, is in `MoDELiB3/ZR3D_GHONIEM_CHANGES.md` and in the companion architecture document.

C Quick reference

The complete sequence from a fresh Windows machine to figures, assuming the prerequisites of Section 2 (Git, Python 3.14, VS Build Tools, CMake, WSL + Ubuntu, SUNDIALS in `Libraries/`) are in place. All commands from the repository root.

```

# 1. clone
PS> git clone https://github.com/Ghoniem/DisloCluster.git ; cd DisloCluster

# 2. python environment + kernel
PS> C:\Python314\python.exe -m venv .DisloClusterVenv
PS> .DisloClusterVenv\Scripts\python.exe -m pip install -r requirements.txt
PS> .DisloClusterVenv\Scripts\python.exe -m ipykernel install --user '
    --name dislocluster --display-name "Python 3.14 (DisloCluster)"

# 3. verify path resolution (repeat after each build)
PS> .DisloClusterVenv\Scripts\python.exe dislocluster_code\paths.py

# 4. build the 0-D solver (native Windows)
PS> cmake -S ZrMicro\cpp_utils -B ZrMicro\build '
    -DCMAKE_BUILD_TYPE=Release
PS> cmake --build ZrMicro\build --config Release

# 5. build the 3-D code (inside WSL; note the explicit MoDELib3 argument)
PS> wsl -u root -e bash Docs/Formulation/build_modelib.sh MoDELib3

# 6. first test: figures from the shipped 3-D results (minutes)
PS> .DisloClusterVenv\Scripts\python.exe '
    dislocluster_code\studies\run_zr3d_singlecrystal.py '
    --doses 1 5 10 30 --max-nm 150

# 7. the full 3-D solve (hours) -- then rerun step 6
PS> wsl -e bash MoDELib3/tutorials/zrmicro_coupled/clean_run.sh

# 8. the 0-D model (minutes)
PS> .DisloClusterVenv\Scripts\python.exe -m nbconvert --to notebook --execute '
    --ExecutePreprocessor.kernel_name=dislocluster '
    --output-dir Simulations\output '
    ZrMicro\code\ZrMicro.ipynb

# 9. the coupled 0-D <-> 3-D march (open interactively, or:)
PS> .DisloClusterVenv\Scripts\python.exe -m nbconvert --to notebook --execute '
    --ExecutePreprocessor.kernel_name=dislocluster '
    --output-dir Simulations\output '
    ZrMicro\code\coupled_0d_3d_ZrMicro.ipynb

```

Key locations:

what	where
path resolution (run it!)	dislocluster_code/paths.py
0-D inputs	Simulations/input/Zr_input_parameters.xlsx
3-D material file	MoDELib3/Library/Materials/Zr3d_ghoniem.txt
3-D case inputs	MoDELib3/tutorials/zrmicro_coupled/inputFiles/
all outputs	Simulations/output/
env overrides	DISLOCLUSTER_ROOT, MODELIB_ROOT, MODELIB_BUILD